Keshav Mahavidyalaya, University of Delhi

Name: Bratadipta Mondal

Course: B.Sc.(H) Computer Science

Roll No: 16021

Subject: Information Security

Practicals Exercise

1. **Demonstrate the use of Network tools: ping, ipconfig, ifconfig, tracert, arp, netstat, whois.**

```
C:\Users\HP>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet 3:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::44a6:247a:9918:592d%7
   IPv4 Address. . . . . . . . . . . : 192.168.56.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Wireless LAN adapter Local Area Connection* 1:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   IPv6 Address. . . . . . . . . . . : 2409:4050:e4f:1475:6c1f:a172:8371:9a9b
   Temporary IPv6 Address. . . . . . : 2409:4050:e4f:1475:e513:cf84:95f3:a312
   Link-local IPv6 Address . . . . . : fe80::f27:de44:58f6:ce41%5
   IPv4 Address. . . . . . . . . . . : 192.168.39.179
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : fe80::f424:22ff:fe81:712%5
                                       192.168.39.7
```

```
C:\Users\HP>ping google.com

Pinging google.com [2404:6800:4002:830::200e] with 32 bytes of data:
Reply from 2404:6800:4002:830::200e: time=296ms
Reply from 2404:6800:4002:830::200e: time=1002ms
Reply from 2404:6800:4002:830::200e: time=920ms
Reply from 2404:6800:4002:830::200e: time=827ms

Ping statistics for 2404:6800:4002:830::200e:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 296ms, Maximum = 1002ms, Average = 761ms
```

```
PS C:\Users\HP> arp -a

Interface: 192.168.39.179 --- 0x5
  Internet Address      Physical Address      Type
  192.168.39.7          f6-24-22-             dynamic
  192.168.39.169        08-00-27-             dynamic
  192.168.39.255        ff-ff-ff-             static
  224.0.0.22            01-00-5e-             static
  224.0.0.251           01-00-5e-             static
  224.0.0.252           01-00-5e-             static
  239.255.255.250       01-00-5e-             static
  255.255.255.255       ff-ff-ff-             static

Interface: 192.168.56.1 --- 0x7
  Internet Address      Physical Address      Type
  192.168.56.255        ff-ff-ff-             static
  224.0.0.22            01-00-5e-             static
  224.0.0.250           01-00-5e-             static
  224.0.0.251           01-00-5e-             static
  224.0.0.252           01-00-5e-             static
  239.255.255.250       01-00-5e-             static
  239.255.255.251       01-00-5e-             static
```

```
C:\Users\HP>tracert google.com

Tracing route to google.com [2404:6800:4002:830::200e]
over a maximum of 30 hops:

  1   595 ms   973 ms  1020 ms  2409:4050:e4f:1475::a1
  2     *        *        *     Request timed out.
  3   157 ms   383 ms   427 ms  2405:200:330:eeee:20::822
  4   567 ms   938 ms   364 ms  2405:200:801:300::e0b
  5     *        *        *     Request timed out.
  6     *        *        *     Request timed out.
  7     *        *        *     Request timed out.
  8     *        *        *     Request timed out.
  9   126 ms  1702 ms   224 ms  2001:4860:1:1::1ef4
 10   803 ms   229 ms   409 ms  2404:6800:81e1:3c0::1
 11   353 ms   373 ms   412 ms  2001:4860:0:1::34f0
 12  1255 ms   852 ms   271 ms  2001:4860:0:1::7598
 13    52 ms   848 ms    73 ms  2001:4860:0:1::78c1
 14   167 ms   972 ms   721 ms  2001:4860:0:1::53cd
 15   425 ms  1376 ms  1287 ms  tzdelb-ap-in-x0e.1e100.net [2404:6800:4002:830::200e]
```

```
C:\Users\HP>netstat -a

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    0.0.0.0:135            DESKTOP-QR7P3NB:0      LISTENING
  TCP    0.0.0.0:445            DESKTOP-QR7P3NB:0      LISTENING
  TCP    0.0.0.0:3306           DESKTOP-QR7P3NB:0      LISTENING
  TCP    0.0.0.0:5040           DESKTOP-QR7P3NB:0      LISTENING
  TCP    0.0.0.0:5357           DESKTOP-QR7P3NB:0      LISTENING
  TCP    0.0.0.0:33060          DESKTOP-QR7P3NB:0      LISTENING
  TCP    0.0.0.0:49664          DESKTOP-QR7P3NB:0      LISTENING
  TCP    0.0.0.0:49665          DESKTOP-QR7P3NB:0      LISTENING
  TCP    0.0.0.0:49666          DESKTOP-QR7P3NB:0      LISTENING
  TCP    0.0.0.0:49667          DESKTOP-QR7P3NB:0      LISTENING
  TCP    0.0.0.0:49668          DESKTOP-QR7P3NB:0      LISTENING
  TCP    0.0.0.0:49673          DESKTOP-QR7P3NB:0      LISTENING
  TCP    127.0.0.1:26590        kubernetes:26591       ESTABLISHED
  TCP    127.0.0.1:26591        kubernetes:26590       ESTABLISHED
  TCP    127.0.0.1:49669        kubernetes:49670       ESTABLISHED
  TCP    127.0.0.1:49670        kubernetes:49669       ESTABLISHED
  TCP    127.0.0.1:49671        kubernetes:49672       ESTABLISHED
  TCP    127.0.0.1:49672        kubernetes:49671       ESTABLISHED
  TCP    127.0.0.1:50534        DESKTOP-QR7P3NB:0      LISTENING
  TCP    127.0.0.1:50564        kubernetes:50565       ESTABLISHED
  TCP    127.0.0.1:50565        kubernetes:50564       ESTABLISHED
  TCP    127.0.0.1:59131        DESKTOP-QR7P3NB:0      LISTENING
  TCP    192.168.39.179:139     DESKTOP-QR7P3NB:0      LISTENING
  TCP    192.168.39.179:21015   51.105.71.137:https    TIME_WAIT
  TCP    192.168.39.179:31391   172.188.155.25:https   ESTABLISHED
  TCP    192.168.39.179:49413   4.213.25.240:https     ESTABLISHED
  TCP    192.168.39.179:61234   98.70.192.224:https    ESTABLISHED
  TCP    192.168.39.179:61235   98.70.192.224:https    ESTABLISHED
```

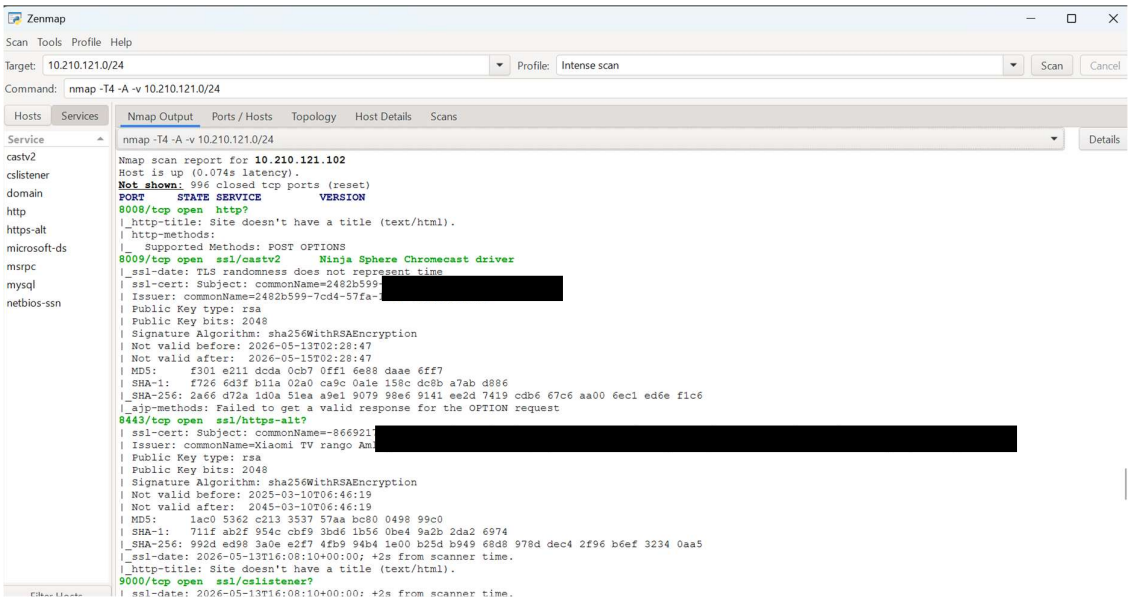
Raw Whois Data

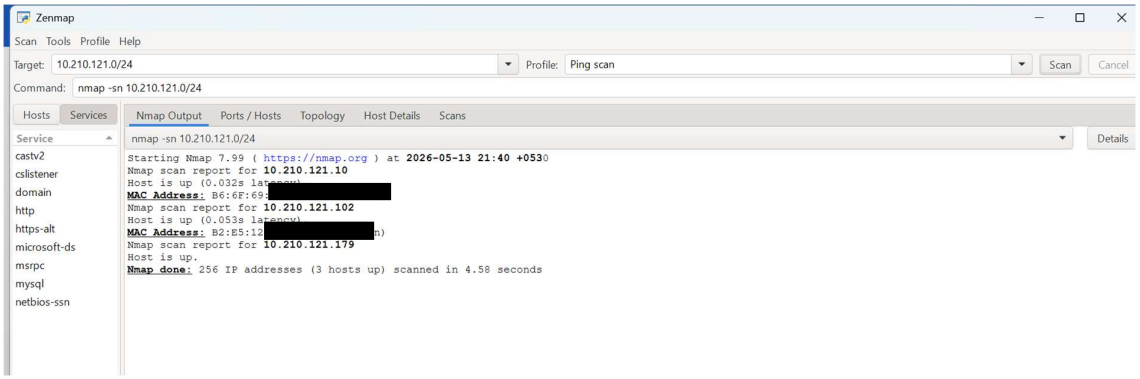
```
Domain Name: du.ac.in
Registry Domain ID: D13836-IN
Registrar URL: http://www.ernet.in
Updated Date: 2024-05-06T12:14:57.903Z
Creation Date: 2004-02-28T05:00:00.039Z
Registry Expiry Date: 2028-02-28T05:00:00.000Z
Registrar: ERNET India
Registrar IANA ID: 800068
Registrar Abuse Contact Email: tejalt@eis.ernet.in
Registrar Abuse Contact Phone: +91.1123358248
Domain Status: clientUpdateProhibited https://icann.org/epp#clientUpdateProhibited
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Registry Registrant ID: REDACTED FOR PRIVACY
Registrant Name: REDACTED FOR PRIVACY
Registrant Organization: Delhi University Computer Centre
Registrant Street: REDACTED FOR PRIVACY
Registrant City: REDACTED FOR PRIVACY
Registrant State/Province:
Registrant Postal Code: REDACTED FOR PRIVACY
Registrant Country: IN
Registrant Phone: REDACTED FOR PRIVACY
Registrant Fax: REDACTED FOR PRIVACY
Registrant Email: Please query the RDDS service of the Registrar of Record identified in this out
```

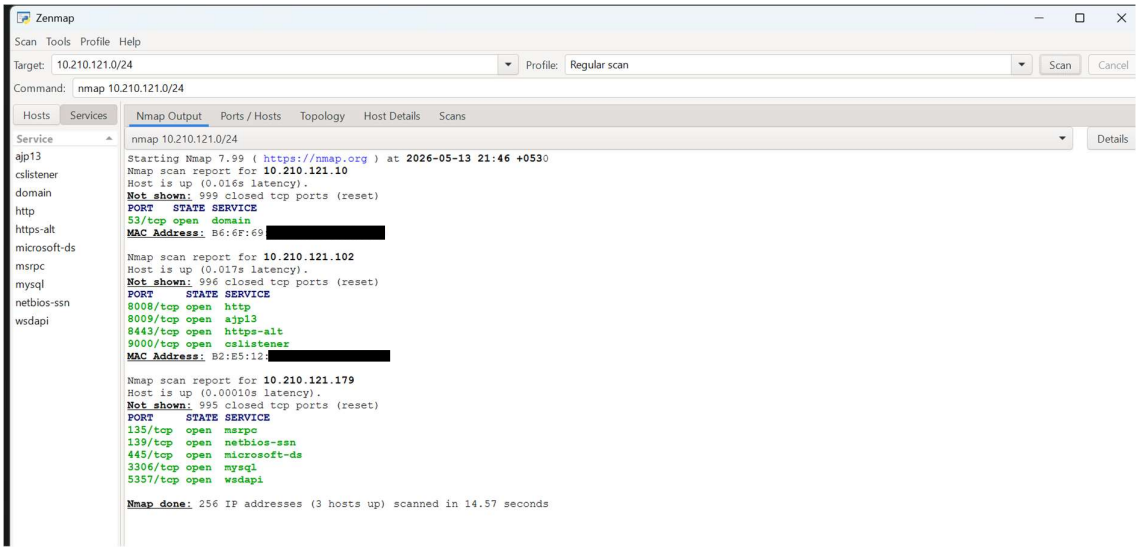
2. **Use of Password cracking tools: John the Ripper, Ophcrack. Verify the strength of passwords using these tools**

```
└─$ john --format=Raw-SHA256 hashes.txt --wordlist=rockyou.txt
Using default input encoding: UTF-8
Loaded 4 password hashes with no different salts (Raw-SHA256 [SHA256 256/256 AVX2 8x])
No password hashes left to crack (see FAQ)

┌──(rbtiger㉿rbtiger)-[~/files]
└─$ john --show --format=Raw-SHA256 hashes.txt
?:password
?:password123
?:deepu
?:12345678

4 password hashes cracked, 0 left
```

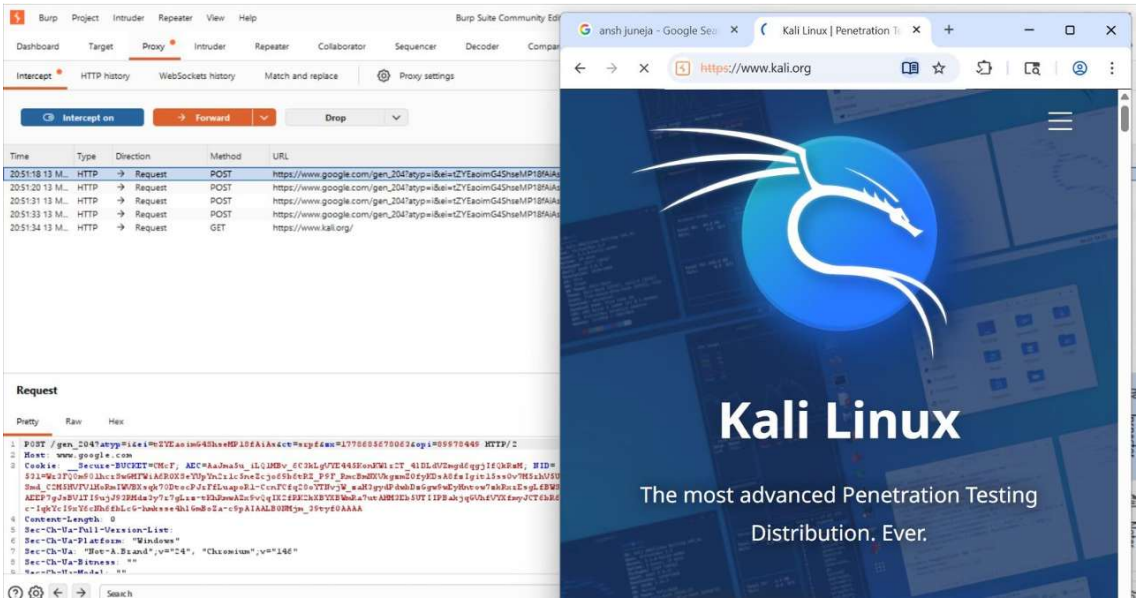
3. **Use nmap/zenmap to analyze a remote machine.**







4. Use Burp proxy to capture and modify the message.



5. Implement caesar cipher substitution operation.

```
def ccipher_encrypt(string:str,key):
    """
    A - 65
    Z - 90
    a - 97
    z - 122
    """
```

```
    # For Key Length Error Handling
    if key > 26:
        while key > 26: key -= 26

    new_str = ""
    for s in string:
        if s == ' ': new_str += s
        elif (ord(s)+key > 65 and ord(s)+key <= 90) or ord(s)+key > 97 and
ord(s)+key <= 122:
            new_str += chr(ord(s)+key)
        elif ord(s)+key > 90:
            new_str += chr(ord(s)+key-26)
        elif ord(s)+key > 122:
            new_str += chr(ord(s)+key-26)
    return new_str

def ccipher_decrypt(string:str,key):
    """
    A - 65
    Z - 90
    a - 97
    z - 122
    """
    # For Key Length Error Handling
    if key > 26:
        while key > 26: key -= 26

    new_str = ""
    for s in string:
        if s == ' ': new_str += s
        elif (ord(s)-key > 64 and ord(s)-key <= 90) or ord(s)-key > 96 and
ord(s)-key <= 122:
            new_str += chr(ord(s)-key)
        elif ord(s)-key < 65:
            new_str += chr(ord(s)-key+26)
        elif ord(s)-key < 97:
            new_str += chr(ord(s)-key+26)
    return new_str


if __name__ == "__main__":
    inp = input("Enter string: ")
    key = int(input("Enter key: "))

    print(f"After caeser cipher encryption: {ccipher_encrypt(inp,key)}")
    print(f"After caeser cipher decryption:
{ccipher_decrypt(ccipher_encrypt(inp,key),key)}")
```

```
PS C:\College works\8th Sem 2026\IS> python -u "c:\College
Enter string: Bratadipta Mondal
Enter key: 5
After caeser cipher encryption: Gwfyfinuyf Rtsifq
After caeser cipher decryption: Bratadipta Mondal
PS C:\College works\8th Sem 2026\IS>
```

6. Implement monoalphabetic and polyalphabetic cipher substitution operation.

Monoalphabetic

```python
import random

def key_generate():
    keys = {}
    for i in range(26):
        char = chr(i+65)
        if char not in keys.keys():
            values = [chr(i+65) for i in range(26)]
            while True:
                r = random.choice(values)
                if r not in keys.values() and r != char:
                    keys[char] = r
                    break
    return keys

def encryption(text):
    cipher = ""
    text = text.upper()
    keys = key_generate()

    for c in text:
        cipher += keys[c]
    return (cipher,keys)

def decryption(keys,cipher):
    text = ""

    for c in cipher:
        text += [key for key,val in keys.items() if c == val][0]

    return text

text = "BratadiptaMondal"
print(f"Normal cipher text: {text}")
```

```
cipher,keys = encryption(text)
print(f"After Encryption text:: {cipher}")
print(f"After Decryption cipher text: {decryption(keys,cipher)}")
```

```
PS C:\College works\8th Sem 2026\IS> python -u "c:\College works\8th Sem 2026\IS\practical6A.py"
Normal cipher text: BratadiptaMondal
After Encryption text:: YSJDJLUKDJHVALJN
After Decryption cipher text: BRATADIPTAMONDAL
PS C:\College works\8th Sem 2026\IS>
```

Polyalphabtic

```
def encryption(text, key):
    cipher = ""
    key = key.upper()
    text = text.upper()

    # Cipher Text: (P + K) mode 26
    for i in range(len(text)):
        cipher += chr((((ord(text[i]) - 65) + (ord(key[i % len(key)]) - 65)) % 26) + 65

    return cipher

def decryption(cipher, key):
    text = ""
    key = key.upper()
    cipher = cipher.upper()

    # Plain Text = (C - K + 26) mode 26
    for i in range(len(cipher)):
        text += chr(
            ((((ord(cipher[i]) - 65) - (ord(key[i % len(key)]) - 65)) + 26) % 26) + 65
        )
```

```
    return text


# text = "BratadiptaMondal"

# key = "Hello"

text = "Hello"

key = "Key"

cipher = encryption(text, key)

plain = decryption(cipher, key)


print(f"Normal text: {text}")

print(f"After encryption cipher text: {cipher}")

print(f"After decryption cipher text: {plain}")
```

```
PS C:\College works\8th Sem 2026\IS> python -u "c:\
Normal text: Hello
After encryption cipher text: RIJVS
After decryption cipher text: HELLO
PS C:\College works\8th Sem 2026\IS>
```

7. Implement playfair cipher substitution operation.

```
import numpy as np

def generate_key_matrix(key):
    key = key.upper()
    key_matrix = [k for k in key]

    for i in range(26):
        char = chr(i+65)
        if (char == "I" and "J" in key_matrix) or (char == "J" and "I" in
key_matrix): continue
        if char not in key_matrix:
            key_matrix.append(char)
```

```python
    key_matrix = np.reshape(key_matrix,(5,5))
    key_matrix[key_matrix == "J"] = "I"
    return key_matrix

def generate_pair(text):
    text = text.upper().replace("J","I")
    if len(text)%2 != 0: text += "X"
    text_pair = []
    for i in range(0,len(text),2):
        if text[i] != text[i+1]: text_pair.append(text[i]+text[i+1])
        else: text_pair.append(text[i]+"X")

    return text_pair

def find_pos(matrix,item):
    return np.argwhere(matrix == item)[0]

def encryption(text,key):
    cipher = ""
    key_matrix = generate_key_matrix(key)
    text_pair = generate_pair(text)

    for txt in text_pair:
        r1,c1 = find_pos(key_matrix,txt[0])
        r2,c2 = find_pos(key_matrix,txt[1])

        if r1 == r2:
            cipher += key_matrix[r1,(c1+1)%5] + key_matrix[r2,(c2+1)%5]
        elif c1 == c2:
            cipher += key_matrix[(r1+1)%5,c1] + key_matrix[(r2+1)%5,c2]
        else:
            cipher += key_matrix[r1,c2] + key_matrix[r2,c1]
    return cipher

def decryption(cipher,key):
    text = ""
    key_matrix = generate_key_matrix(key)
    cipher_pair = generate_pair(cipher)

    for cp in cipher_pair:
        r1,c1 = find_pos(key_matrix,cp[0])
        r2,c2 = find_pos(key_matrix,cp[1])

        if r1 == r2:
            text += key_matrix[r1,(c1-1)%5] + key_matrix[r2,(c2-1)%5]
        elif c1 == c2:
            text += key_matrix[(r1-1)%5,c1] + key_matrix[(r2-1)%5,c2]
```

```
        else:
            text += key_matrix[r1,c2] + key_matrix[r2,c1]

    pair = generate_pair(text)
    new_text = ""
    for p in pair:
        if p[1] == "X": new_text += p[0]+p[0]
        else: new_text += p
    if text.endswith("X"):
        new_text = new_text[:-1]
    return new_text

key = "MONARCHY"
text = "BrataaiptaM"
cipher = encryption(text,key)
plain = decryption(cipher,key)
print(f"Normal Text: {text}, with key: \n{generate_key_matrix(key)}")
print(f"After encryption cipher text: {cipher}")
print(f"After decryption text: {plain}")
```

```
PS C:\College works\8th Sem 2026\IS> python -u "c:\College works\8th Sem 2026\IS\practical7.py"
Normal Text: BrataaiptaM, with key:
[['M' 'O' 'N' 'A' 'R']
 ['C' 'H' 'Y' 'B' 'D']
 ['E' 'F' 'G' 'I' 'K']
 ['L' 'P' 'Q' 'S' 'T']
 ['U' 'V' 'W' 'X' 'Z']]
After encryption cipher text: DARSBAFSSRAU
After decryption text: BRATAAIPTAM
PS C:\College works\8th Sem 2026\IS>
```

8. Implement hill cipher substitution operation.

```
import numpy as np
import string
import re
import sympy as sp

def hCrypt(n,text,key):
    if n**2 != len(key):
        return False

    alphabets = string.ascii_uppercase
    text_matrix = []
    key_matrix = []

    for k in key: key_matrix.append(alphabets.index(k))
```

```
    for t in text: text_matrix.append(alphabets.index(t))

    key_matrix = np.reshape(key_matrix,(n,n))
    dot_key_txt = np.dot(key_matrix,text_matrix)%26

    cipher = ""
    for d in dot_key_txt:
        cipher += alphabets[d]
    return cipher

def encryption(text,key):
    n = 3 # Block Size

    text = text.upper()
    key = key.upper()

    if len(text)%n < n:
        text += "X"*(n-(len(text)%n))

    txt_pair = [text[i:i+n] for i in range(0,len(text),n)]

    cipher = ""
    for p in txt_pair:
        cipher += hCrypt(n,p,key)
    return cipher


def hDecrypt(n,text,key):
    if n**2 != len(key): return False

    alphabets = string.ascii_uppercase
    text_matrix = []
    key_matrix = []

    for t in text: text_matrix.append(alphabets.index(t))
    for k in key: key_matrix.append(alphabets.index(k))

    key_matrix = sp.Matrix(n,n,key_matrix)
    det_key = int(key_matrix.det())%26

    if sp.gcd(det_key,26) != 1: return None

    mod_inverse_key = sp.mod_inverse(det_key,26)
    inv_key_matrix = (mod_inverse_key*key_matrix.adjugate())%26

    dot_key_matrix = (np.dot(inv_key_matrix,text_matrix).astype(int))%26

    cipher = ""
    for d in dot_key_matrix:
```

```
        cipher += alphabets[d]
    return cipher

def decryption(cipher,key):
    n = 3 # Block

    cipher = cipher.upper()
    key = key.upper()

    txt_pair = [cipher[i:i+n] for i in range(0,len(cipher),n)]

    text = ""
    for t in txt_pair:
        text += hDecrypt(n,t,key)

    text = re.sub(r"X+$","",text)
    return text



text = "BratadiptaMondal"
key = "abcdewxyz"
cipher = encryption(text,key)
plain = decryption(cipher,key)

print(f"Plain text: {text}")
print(f"After encryption cipher text: {cipher}")
print(f"After decryption plain text: {plain}")
```

```
PS C:\College works\8th Sem 2026\IS> python -u "c:\College works\8th Sem 2026\IS\prcatical8_hill_cipher.py"
Plain text: BratadiptaMondal
After encryption cipher text: RTPGTSBIFOSODZHRHC
After decryption plain text: BRATADIPTAMONDAL
PS C:\College works\8th Sem 2026\IS>
```

9. Implement rail fence cipher transposition operation.

```
import numpy as np

def rfEncryption(text,key):
    txt_len = len(text)
    flag = False
    row = 0
    matrix = np.full((key,txt_len),'\n')
    for i in range(txt_len):
        if row == key-1 or row == 0:
            flag = not flag
```

```
        if flag:
            matrix[row][i] = text[i]
            row += 1
        else:
            matrix[row][i] = text[i]
            row -= 1
    cipher = ''
    for m in matrix:
        cipher += ''.join(i for i in m if i != '\n')
    return cipher

def rfDeccryption(cipher,key):
    ciphr_len = len(cipher)
    flag = False
    row = 0
    matrix = np.full((key,ciphr_len),'\n')
    for i in range(ciphr_len):
        if row == key-1 or row == 0:
            flag = not flag
        if flag:
            matrix[row][i] = '*'
            row += 1
        else:
            matrix[row][i] = '*'
            row -= 1

    k=0
    for i in range(key):
        for j in range(ciphr_len):
            if matrix[i][j] == '*':
                matrix[i][j] = cipher[k]
                k+=1

    flag = False
    row = 0
    txt = ''
    for i in range(ciphr_len):
        if row == key-1 or row == 0:
            flag = not flag
        if flag:
            txt += matrix[row][i]
            row += 1
        else:
            txt += matrix[row][i]
            row -= 1
    return txt

key = 5
```

```python
cipher = rfEncryption("Bratadipta Mondal",key)
print(f"Cipher: {cipher} with key: {key}")
print(f"Normal Text: {rfDeccryption(cipher,key)}")
```

```
PS C:\College works\8th Sem 2026\IS> python -u "c:\College works\8th Sem 2026\IS\is cipher\rail fense.py"
Cipher: Btlrpaaai dtdMnao with key: 5
Normal Text: Bratadipta Mondal
PS C:\College works\8th Sem 2026\IS>
```

10. Implement row transposition cipher transposition operation.

```python
def cipher_encryption():
    msg = input("Enter Plain Text: ").replace(" ", "").upper()
    # print(msg)
    key = input("Enter keyword: ").upper()

    # assigning numbers to keywords
    kywrd_num_list = keyword_num_assign(key)

    # printing key
    for i in range(len(key)):
        print(key[i], end=" ", flush=True)

    print()
    for i in range(len(key)):
        print(str(kywrd_num_list[i]), end=" ", flush=True)
    print()


    # in case characters don't fit the entire grid perfectly.
    extra_letters = len(msg) % len(key)
    # print(extraLetters)
    dummy_characters = len(key) - extra_letters
    # print(dummyCharacters)

    if extra_letters != 0:
        for i in range(dummy_characters):
            msg += "."


    # print(msg)

    num_of_rows = int(len(msg) / len(key))

    # Converting message into a grid
    arr = [[0] * len(key) for i in range(num_of_rows)]
```

```
    z = 0
    for i in range(num_of_rows):
        for j in range(len(key)):
            arr[i][j] = msg[z]
            z += 1

    for i in range(num_of_rows):
        for j in range(len(key)):
            print(arr[i][j]*2, end=" ", flush=True)
        print()

    # getting locations of numbers
    num_loc = get_number_location(key, kywrd_num_list)

    print(num_loc)

    # cipher
    cipher_text = ""

    for k in range(len(key)):  # Loop through each column in the sorted order
        d = int(num_loc[k])    # Get the actual index of the column to read
        for j in range(num_of_rows): # Loop down through every row of that
column
            cipher_text += arr[j][d]

    print("Cipher Text: {}".format(cipher_text))


def get_number_location(key, kywrd_num_list):
    num_loc = ""
    for i in range(len(key) + 1):
        for j in range(len(key)):
            if kywrd_num_list[j] == i:
                num_loc += str(j)

    return num_loc


def keyword_num_assign(key):
    alpha = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    kywrd_num_list = list(range(len(key)))
    # print(kywrdNumList)
    init = 0
    for i in range(len(alpha)):
        for j in range(len(key)):
            if alpha[i] == key[j]:
                init += 1
                kywrd_num_list[j] = init
```

```python
    return kywrd_num_list


def cipher_decryption():
    msg = input("Enter Cipher Text: ").replace(" ", "").upper()
    # print(msg)
    key = input("Enter keyword: ").upper()

    # assigning numbers to keywords
    kywrd_num_list = keyword_num_assign(key)

    num_of_rows = int(len(msg) / len(key))

    # getting locations of numbers
    num_loc = get_number_location(key, kywrd_num_list)

    # Converting message into a grid
    arr = [[0] * len(key) for i in range(num_of_rows)]

    # decipher
    plain_text = ""
    k = 0
    itr = 0

    for i in range(len(msg)):
        d = 0
        if k == len(key):
            k = 0
        else:
            d: int = int(num_loc[k])
        for j in range(num_of_rows):
            arr[j][d] = msg[itr]
            itr += 1
        if itr == len(msg):
            break
        k += 1
    print()

    for i in range(num_of_rows):
        for j in range(len(key)):
            plain_text += str(arr[i][j])


    print("Plain Text: " + plain_text)


if __name__ == "__main__":
    choice = int(input("1. Encryption\n2. Decryption\nChoose(1,2): "))
```

```
    if choice == 1:
        print("Encryption")
        cipher_encryption()
    elif choice == 2:
        print("Decryption")
        cipher_decryption()
    else:
        print("Invalid Choice")
```

```
PS C:\College works\8th Sem 2026\IS> python -u "c:\College works\8th Sem 2026\IS\is cipher\row _transposition.py"
1. Encryption
2. Decryption
Choose(1,2): 1
Encryption
Enter Plain Text: Bratadipta Mondal
Enter keyword: hello
H E L L O
2 1 3 4 5
B R A T A
D I P T A
M O N D A
L . . . .
10234
Cipher Text: RIO.BDMLAPN.TTD.AAA.
PS C:\College works\8th Sem 2026\IS>
```

11. Implement product cipher transposition operation.

```
import numpy as np
import random

# Substitution
def encryption(text, key):
    cipher = ""
    key = key.upper()
    text = text.upper()

    # Cipher Text: (P + K) mode 26
    for i in range(len(text)):
        cipher += chr((((ord(text[i]) - 65) + (ord(key[i % len(key)]) - 65)) %
26) + 65)

    return cipher


def decryption(cipher, key):
    text = ""
    key = key.upper()
    cipher = cipher.upper()

    # Plain Text = (C - K + 26) mode 26
    for i in range(len(cipher)):
```

```
        text += chr(
            ((((ord(cipher[i]) - 65) - (ord(key[i % len(key)]) - 65)) + 26) %
26) + 65
        )

    return text

# Transposition
def rfEncryption(text,key):
    txt_len = len(text)
    flag = False
    row = 0
    matrix = np.full((key,txt_len),'\n')
    for i in range(txt_len):
        if row == key-1 or row == 0:
            flag = not flag

        if flag:
            matrix[row][i] = text[i]
            row += 1
        else:
            matrix[row][i] = text[i]
            row -= 1
    cipher = ''
    for m in matrix:
        cipher += ''.join(i for i in m if i != '\n')
    return cipher

def rfDeccryption(cipher,key):
    ciphr_len = len(cipher)
    flag = False
    row = 0
    matrix = np.full((key,ciphr_len),'\n')
    for i in range(ciphr_len):
        if row == key-1 or row == 0:
            flag = not flag
        if flag:
            matrix[row][i] = '*'
            row += 1
        else:
            matrix[row][i] = '*'
            row -= 1

    k=0
    for i in range(key):
        for j in range(ciphr_len):
            if matrix[i][j] == '*':
                matrix[i][j] = cipher[k]
```

```
                k+=1

    flag = False
    row = 0
    txt = ''
    for i in range(ciphr_len):
        if row == key-1 or row == 0:
            flag = not flag
        if flag:
            txt += matrix[row][i]
            row += 1
        else:
            txt += matrix[row][i]
            row -= 1
    return txt

def product_encrypt(text,key):
    # Substitution
    cipher = encryption(text,key)

    # Transposition
    cipher = rfEncryption(cipher,len(key))

    return cipher

def product_decrypt(cipher,key):
    # Transposition
    text = rfDeccryption(cipher,len(key))

    # Substitution
    text = decryption(text,key)
    return text


key = "hello"
cipher = product_encrypt("Bratadipta Mondal",key)
print(f"Cipher: {cipher} with key: {key}")
print(f"Normal Text: {product_decrypt(cipher,key)}")
```

```
PS C:\College works\8th Sem 2026\IS> python -u "c:\College works\8th Sem 2026\IS\practical11.py"
Cipher: IEPVAOHLMAREKQYOZ with key: hello
Normal Text: BRATADIPTATMONDAL
PS C:\College works\8th Sem 2026\IS>
```